

Министерство науки и высшего образования Российской
Федерации
Федеральное государственное бюджетное образовательное
учреждение высшего образования
«Московский государственный технический университет имени
Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 5

по дисциплине «Анализ Алгоритмов»

Тема Организация параллельных вычислений по конвейерному принципу

Студент Поляков А.И.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л.

Москва, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Входные и выходные данные	4
2 Преобразование входных данных в выходные	5
3 Примеры работы программы	7
4 Тестирование	8
5 Описание исследования	9
ЗАКЛЮЧЕНИЕ	10
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	11

ВВЕДЕНИЕ

Параллелизм описывает одновременное выполнение различных последовательностей действий [1]. Следовательно, параллельные вычисления подразумевают выполнение операций одновременно. Распараллеливание вычислений может повысить временную эффективность программы на многопроцессорных системах и даже на однопроцессорных, если программа часто простаивает, ожидая ввода-вывода [2].

Некоторые программы можно разделить на отдельные компоненты, каждая из которых выполняет определённую операцию над данными и передаёт результат следующему компоненту. В результате формируется конвейер обработки данных, где элементы этого конвейера могут функционировать параллельно друг с другом.

Цель данной работы – разработка программного обеспечения для обработки файлов страниц с сайта vkusnoprigotovim.ru.

Задачи работы включают разработку ПО, выполняющего обработку файлов рецептов в конвейере и исследование временных характеристик разработанной программы.

1 Входные и выходные данные

Входными данными для программы является набор файлов с информацией о рецептах. Выходными данными является информация о рецептах, записанная в хранилище.

2 Преобразование входных данных в выходные

Для реализации ПО был использован язык Golang [3] с использованием корутин для организации конвейера.

Конвейер состоит из 3-х частей:

- 1) чтение данных из файла;
- 2) извлечение необходимого подмножества данных;
- 3) запись извлеченных данных в хранилище.

Каждая из частей запускается в отдельной корутине [4], которые передают данные через каналы [4].

Элементом данных выступает структура, которая хранит в себе информацию о рецепте, информацию о моментах времени начала и конца обработки отдельной частью программы и метаданные для обработчиков конвейера. Структура представлена в листинге 2.1.

Листинг 2.1 – Данные, передающиеся между частями конвейера

```
type Task struct {
    ID                int
    FileName          string
    Text              string
    Data              Data
    CreationTime      time.Time
    QueueTimes        []time.Time
    ProcessingTimes    []time.Time
    CompletionTime    time.Time
    Context            *TaskContext
}

type Data struct {
    Id                int           `db:"id"`
    Url               string        `db:"url"`
    Title             string        `db:"title"`
    Ingredients        []Ingredient  `db:"ingredients"`
    Steps              []string      `db:"steps"`
    ImageUrl          string        `db:"image_url"`
}

type Ingredient struct {
```

```
    Name      string 'json:"name"'
    Unit      string 'json:"unit"'
    Quantity  string 'json:"quantity"'
}

type TaskContext struct {
    Dsn string
}
```

Полученные в результате работы программы рецепты сохраняются в базе данных PostgreSQL [5], а из неё отдельным скриптом могут быть сохранены в виде json-файлов.

3 Примеры работы программы

В листинге 3.1 представлен пример рецепта, обработанного программой.

Листинг 3.1 – Данные рецепта

```
{
  "id": 498,
  "issue_id": 9170,
  "url":
    "https://vkusnoprigotovim.ru/zagotovki/zharenye-baklazhany",
  "title": "Жареные баклажаны на зиму",
  "ingredients": [
    {
      "name": "баклажаны - по вкусу",
      "unit": "",
      "quantity": ""
    }
  ],
  "steps": [
    "Баклажаны почистить и порезать длинненькими кусочками.",
    "Через 2 часа воду слить и промыть баклажаны.",
    "Обжаренные баклажаны выложить на салфетку и положить в морозильник.",
    "Зимой это хороший способ разнообразить свое меню."
  ],
  "image_url":
    "/zagotovki/zharenye-baklazhany-na-zimu/zharenye-baklazhany.jpg"
}
```

4 Тестирование

Тестирование программы проводилось загрузкой в качестве входных данных 500 файлов различных страниц рецептов. При этом отслеживалось количество удачных обработок данных, а также выборочная ручная проверка 10 случайных рецептов.

Тестирование было успешно пройдено.

5 Описание исследования

Технические характеристики устройства, на котором проводились замеры:

- процессор: AMD Ryzen 7 5800U (16) @ 4.51 ГГц;
- оперативная память: 16 Гб;
- ядро ОС Linux 6.11.6-zen1-1-zen.

При проведении замеров ноутбук был включён в сеть и были запущены только системные приложения.

На вход программе были поданы 500 файлов, каждый из которых был успешно обработан.

По итогу выполнения программы выводятся:

- среднее время существования задачи;
- среднее время ожидания задачи в каждой из очередей;
- среднее время обработки задачи на каждой из стадий.

В листинге 5.1 представлены результаты замеров.

Листинг 5.1 – Данные журналирования

```
Average task lifetime: 1.379249857s
Average queue time for Stage 1: 323.860945ms
Average queue time for Stage 2: 458.44184ms
Average queue time for Stage 3: 590.42342ms
Average processing time for Stage 1: 31.578mks
Average processing time for Stage 2: 66.874mks
Average processing time for Stage 3: 6.490491ms
```

Самым долгим этапом обработки файлов оказался этап преобразования данных. Самым быстрым - этап записи в хранилище.

ЗАКЛЮЧЕНИЕ

Было разработано программное обеспечение, выполняющее обработку файлов страниц с сайта vkusnoprigitovim.ru.

В ходе работы были решены следующие задачи:

- разработка ПО, выполняющего обработку файлов рецептов в конвейере;
- исследование временных характеристик разработанной программы.

В результате исследования ПО, наиболее продолжительным этапом обработки файлов оказался этап преобразования данных, а самым быстрым - этап записи в базу данных.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. David R. Butenhof Programming with POSIX threads [Текст] / David R. Butenhof — Washington: Library of Congress Cataloging-in-Publication Data, 1997 — 202 с.
2. Таненбаум Э., Бос Х. Современные операционные системы. [Текст] / Таненбаум Э., Бос Х. — 4-е изд.. — СПб.: Питер, 2015 — 1120 с.
3. GO // GO programming language URL: <https://go.dev/> (дата обращения: 24.10.2024).
4. The Go Memory Model // GO programming language URL: <https://go.dev/ref/mem> (дата обращения: 24.10.2024).
5. PostgreSQL: The World's Most Advanced Open Source Relational Database // PostgreSQL URL: <https://www.postgresql.org/> (дата обращения: 24.10.2024).